

Adversarial Attacks

Robust Deep Learning (Seminar)

Bjarne Thal Marius Wrobel

Dept. of Computer Science, RWTH Aachen University, Germany
`{bjarne.thal, marius.wrobel}@rwth-aachen.de`

August 4, 2026



Which one is the monkey?



Which one is the monkey?



panda



monkey

Adversarial Examples



“panda”
57.7% confidence

+ .007 ×



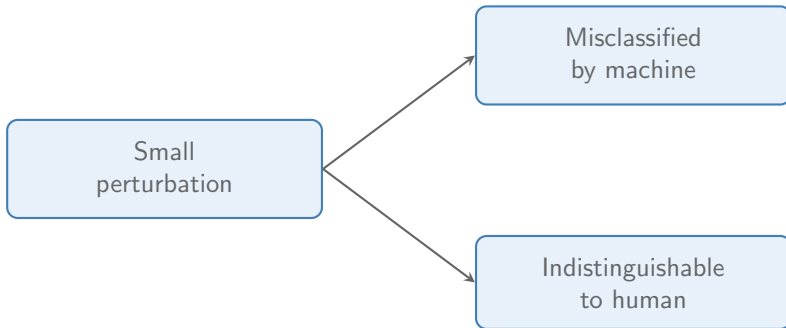
=



“gibbon”
99.3 % confidence

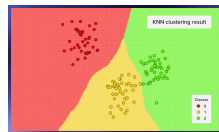
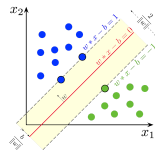
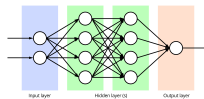
I. J. Goodfellow, Shlens, and Szegedy 2014

Adversarial Examples



Transferability

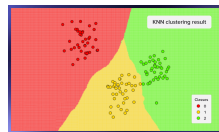
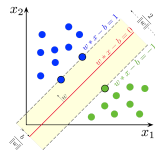
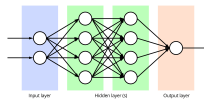
- ▶ Attacks transfer across architectures, training data, and more



Papernot, McDaniel, and I. Goodfellow 2016

Transferability

- ▶ Attacks transfer across architectures, training data, and more
- Adv. ex. are properties of a problem itself



Papernot, McDaniel, and I. Goodfellow 2016

Why this matters

- ▶ **Fundamental blind spot** while integrated AI systems require high reliability



Images: medicalimaging.na; thetravelimages.com

Outline

1. Evasion attacks
2. Poisoning attacks
3. Discussion
 - 3.1 Explaining adversarial examples
 - 3.2 Adversarial attacks as a subject of study

Outline

1. Evasion attacks
2. Poisoning attacks
3. Discussion

Taxonomy & Terminology

Attacker knowledge

White-box

vs.

Black-box

Attack stage

Evasion (at test time)

vs.

Poisoning (at training time)

Attack goal

Targeted (specific wrong label)

vs.

Untargeted (any wrong label)

L-BFGS

$$\min_{\delta} \|\delta\|_2 \quad \text{subject to} \quad f(x + \delta) = t$$

- ▶ f has no gradient — it returns a discrete class label, $f(x + \delta) = t$ is a hard, non-differentiable condition
 - ▶ Derivative-based optimisers such as L-BFGS need a gradient to move at all
- ↪ Reformulate

$$\min_{\delta} c \|\delta\|_2 + L_f(x + \delta, t)$$

Significance

- ▶ First method to **reliably generate** adversarial examples
- ▶ Computationally expensive — far too slow to be practical
- ↪ It *proved a point* rather than providing a usable attack

Fast Gradient Sign Method (FGSM)

Fast Gradient Sign Method

I. J. Goodfellow, Shlens, and Szegedy 2014

White-box · Evasion · Untargeted

- ▶ Previously: Adversarial examples assumed to be caused by 'complex' behaviour
- ▶ Goodfellow et al.: Linear models have adversarial examples as well
- Models may be too linear

High-dimensional linearity

I. J. Goodfellow, Shlens, and Szegedy 2014

- ▶ Linear model $w^\top x$ perturbed by δ , $\|\delta\|_\infty \leq \epsilon$: $w^\top x + \delta = w^\top x + w^\top \delta$

High-dimensional linearity

I. J. Goodfellow, Shlens, and Szegedy 2014

- ▶ Linear model $w^\top x$ perturbed by δ , $\|\delta\|_\infty \leq \epsilon$: $w^\top x + \delta = w^\top x + w^\top \delta$
- ▶ Perturbation shifts activation by $w^\top \delta$

High-dimensional linearity

I. J. Goodfellow, Shlens, and Szegedy 2014

- ▶ Linear model $w^\top x$ perturbed by δ , $\|\delta\|_\infty \leq \epsilon$: $w^\top x + \delta = w^\top x + w^\top \delta$
- ▶ Perturbation shifts activation by $w^\top \delta$
- ▶ Best strategy: δ aligned with the weights, then: $w^\top \delta = \epsilon \sum_{i=1}^n |w_i| = \epsilon m n$

High-dimensional linearity

I. J. Goodfellow, Shlens, and Szegedy 2014

- ▶ Linear model $w^\top x$ perturbed by δ , $\|\delta\|_\infty \leq \epsilon$: $w^\top x + \delta = w^\top x + w^\top \delta$
- ▶ Perturbation shifts activation by $w^\top \delta$
- ▶ Best strategy: δ aligned with the weights, then: $w^\top \delta = \epsilon \sum_{i=1}^n |w_i| = \epsilon m n$
- ▶ Score change scales *linearly* with number of dimensions

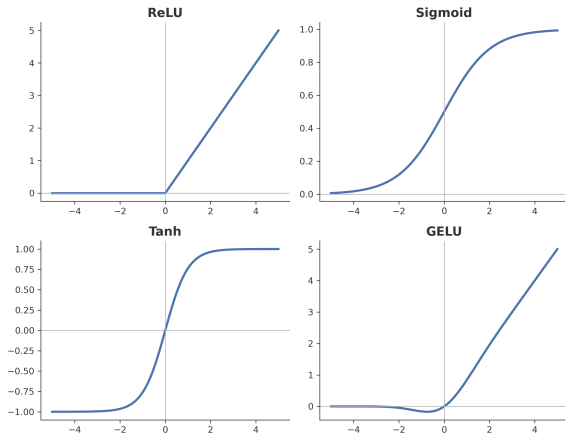
High-dimensional linearity

I. J. Goodfellow, Shlens, and Szegedy 2014

- ▶ Linear model $w^\top x$ perturbed by δ , $\|\delta\|_\infty \leq \epsilon$: $w^\top x + \delta = w^\top x + w^\top \delta$
 - ▶ Perturbation shifts activation by $w^\top \delta$
 - ▶ Best strategy: δ aligned with the weights, then: $w^\top \delta = \epsilon \sum_{i=1}^n |w_i| = \epsilon m n$
 - ▶ Score change scales *linearly* with number of dimensions
- ↪ Many imperceptible changes accumulate into one large shift

Do Neural Networks behave linearly?

I. J. Goodfellow, Shlens, and Szegedy 2014



Fast Gradient Sign Method

I. J. Goodfellow, Shlens, and Szegedy 2014

White-box · Evasion · Untargeted

- ▶ Linearise the loss function:

$$\arg \max_{\|\delta\|_{\infty} \leq \epsilon} L_f(x + \delta, y) = \arg \max_{\|\delta\|_{\infty} \leq \epsilon} L_f(x, y) + \nabla_x L_f(x, y)^T \delta$$

- ▶ Maximised by $\delta = \epsilon \cdot \text{sign}(\nabla_x L_f(x, y))$

↪ Single step in gradient direction

FGSM: Significance

I. J. Goodfellow, Shlens, and Szegedy 2014

White-box · Evasion · Untargeted

- ▶ Much faster than L-BFGS: Single step only
- ↪ Made adversarial training feasible
- ▶ Introduced different way of thinking about adversarial examples

Projected Gradient Descent (PGD)

General formulation of adversarial training. Adversary solves

$$\max_{\delta \in S} L_f(x + \delta, y)$$

- ▶ FGSM: Use $S = \{\delta \mid \|\delta\|_{\infty} \leq \epsilon\}$
- ▶ Use projected gradient descent to optimise

PGD - a general first-order attack?

Madry et al. 2017

White-box · Evasion · Untargeted

- ▶ Random runs show different maxima, but similar loss values
- ↪ If better solution exists: difficult to find using first-order methods
- ▶ Especially relevant for adversarial defenses

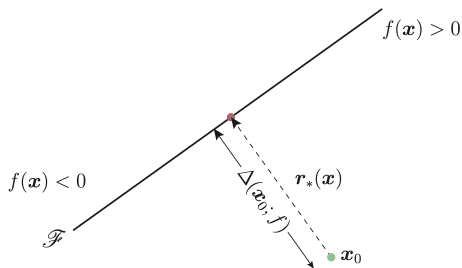
DeepFool

DeepFool

Moosavi-Dezfooli, Fawzi, and Frossard 2016 — the affine, binary case

White-box · Evasion · Untargeted

- ▶ Decision boundary is a *hyperplane*
- ▶ Minimal perturbation is *orthogonal projection* of x onto $\mathcal{F} \rightarrow$ closed form

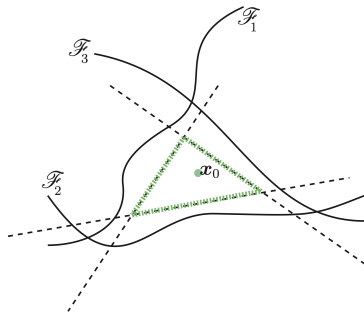


DeepFool

Moosavi-Dezfooli, Fawzi, and Frossard 2016 — the general, multiclass case

White-box · Evasion · Untargeted

- ▶ **Non-linear:** iterative $\rightarrow$ linearise the score function at current point (first-order Taylor), project onto that hyperplane, repeat until label flips
- ▶ **Multiclass:** Linearise every class boundary, project onto the **closest**



Significance

- ▶ **Geometric reading:** robustness as distance to the decision boundary

Transfer attacks

Transfer attacks

- ▶ Black-box attack pattern
- ▶ Adversary creates and trains a substitute model for the target
- ▶ Adversarial examples for substitute model are found (white-box!)
- ▶ These can be used to attack the target

Jacobian-Based Dataset Augmentation

Papernot, McDaniel, I. Goodfellow, et al. 2017

Black-box · Evasion · -

- ▶ Assumption: No knowledge of model parameters, dataset
Access to somewhat representative inputs is granted (e.g. natural images)
- ▶ How do we construct a suitable substitute? (similar decision boundaries)
- ▶ Construct an artificial dataset!

Jacobian-Based Dataset Augmentation

Papernot, McDaniel, I. Goodfellow, et al. 2017

Black-box · Evasion · -

- ▶ Use target model as an oracle to label samples
- ▶ Strategy for sample selection is necessary
- ▶ Idea: iteratively add meaningful samples to dataset
- ▶ Use Jacobian for a heuristic

Heuristic for meaningful directions

Papernot, McDaniel, I. Goodfellow, et al. 2017

Black-box · Evasion · -

$$x = x + \alpha \cdot \text{sign}(J_f(x)[o(x)])$$

- ▶ $J_f(x)$: total derivative
- ▶ $J_f(x)[o(x)]$: total derivative, column of the correct label
- We move into the class region (according to the substitute)!

Jacobian-Based Dataset Augmentation

Papernot, McDaniel, I. Goodfellow, et al. 2017

Black-box · Evasion · -

- ▶ 1) Collect samples somewhat related to the input domain
- ▶ 2) Choose architecture for substitute
- ▶ 3) Target model labels current samples
- ▶ 4) Train substitute
- ▶ 5) Expand dataset: $S_{N+1} := \{x = x + \alpha \cdot \text{sign}(J_f(x)[o(x)]) \mid x \in S_N\} \cup S_N$
- ▶ Repeat (3) onwards

ZOO

ZOO: Zeroth order black-box attacks

chen2017zoo

Black-box · Evasion · Targ./Untarg.

- ▶ Black-box attack without substitute model
- ▶ No quality loss due to transfer
- ▶ Assumptions: Adversary allowed to query, gets confidence scores

$$\hat{g}_i := \frac{\partial l(x)}{\partial x_i} \approx \frac{l(x + he_i) - l(x - he_i)}{2h}$$

- ▶ l loss function based on confidence scores
- ▶ Two queries per coordinate, full gradient infeasible
- ↪ Coordinate descent

Outline

1. Evasion attacks
2. Poisoning attacks
3. Discussion

Hidden Trigger Backdoor Attack

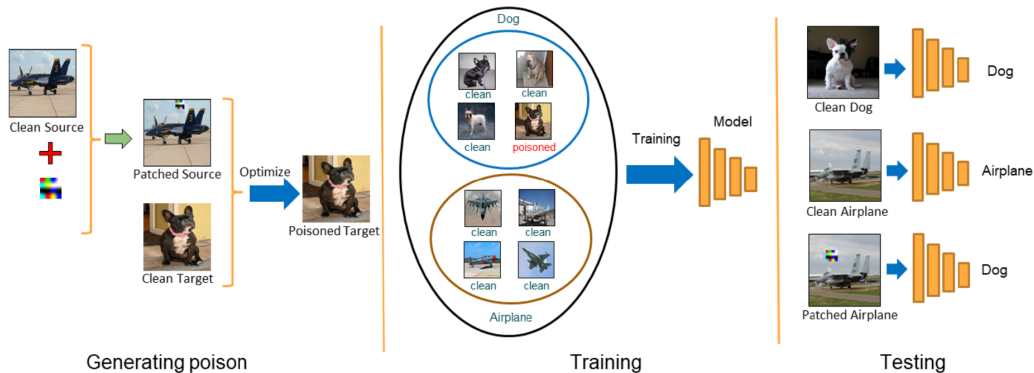
Hidden Trigger Backdoor Attack

saha2020hidden

White-box · Poisoning · Targeted

- ▶ Goal: get ML model to misclassify any sample with trigger pattern into target class t
- ▶ Assumption: model based on pretrained weights known to the attacker, access to data distribution
- ▶ Restriction: adversary cannot choose labels ("Clean-label attack")

Hidden Trigger Backdoor Attack: The approach



Taken from [saha2020hidden](#).

- ▶ Sample instances s b of target class t
- ▶ Apply trigger to source: patched source $\tilde{s}$
- ▶ Poison instances:

$$\arg \min_x \|g(x) - g(\tilde{s})\|_2^2 \quad \textbf{s.t.} \quad \|x - b\|_\infty < \epsilon$$

- ▶ g : feature representation (second-to-last layer output)
- ▶ Keep x close to $\tilde{s}$ in feature space, but close to b in pixel space

Narcissus

- ▶ Goal: get ML model to misclassify any sample with trigger pattern into target class t
- ▶ Assumption: no access to model or data distribution
- ▶ But: Access to "public out of distribution" data D_{POOD} , target class examples D_t
- ▶ Restriction: adversary cannot choose labels ("clean-label attack")
- ▶ Now: Optimise the trigger pattern!

- ▶ Assume oracle f_{orc} , which is the victim's model without poisoning
- ▶ Choose trigger to maximise oracle's confidence on target class:
$$\delta^* = \arg \min_{\delta \in S} \sum_{(x,t) \in D_t} L_{f_{orc}}(x + \delta, t)$$
- ▶ Trigger now is related to the target class!
- ▶ However: No access to f_{orc}

- ▶ Train surrogate f_{sur} on D_{POOD} , D_t
- ▶ Use PGD to find $\delta^* = \arg \min_{\delta \in S} \sum_{(x,t) \in D_t} L_{f_{sur}}(x + \delta, t)$
- ▶ Add trigger to portion of D_t , give to victim
- ▶ Exploit: Add trigger to desired input

Outline

1. Evasion attacks
2. Poisoning attacks
3. Discussion
 - 3.1 Explaining adversarial examples
 - 3.2 Adversarial attacks as a subject of study

Explaining adversarial examples

2013 Szegedy et al. — quirks of non-linearity

Explaining adversarial examples

2013 Szegedy et al. — quirks of non-linearity

2014 Goodfellow et al. — high-dimensional linearity

Explaining adversarial examples

2013 **Szegedy et al.** — quirks of non-linearity

2014 **Goodfellow et al.** — high-dimensional linearity

2019 **Ilyas et al.** — predictive but non-robust features

Predictive and robust features

Ilyas et al. 2019

Predictive

Correlated with the label in expectation.

Predictive and robust features

Ilyas et al. 2019

Predictive

Correlated with the label in expectation.

Robust

Stays correlated *under* adversarial perturbation.

Predictive and robust features

Ilyas et al. 2019

Predictive

Correlated with the label in expectation.

Robust

Stays correlated *under* adversarial perturbation.

→ **Predictive but non-robust:** Standard training happily relies on such features

Experiment: "Misabeled" training dataset

Ilyas et al. 2019



Real-life adversarial attacks

- ▶ Real confirmed malicious attacks are surprisingly rare
- ▶ Two candidate explanations:
 1. They slip through unnoticed, logged as ordinary misclassifications
 2. They are simply of little use to actual attackers

Adversarial attacks as a subject of study

- ▶ Not really significant security threat

Adversarial attacks as a subject of study

- ▶ Not really significant security threat
- ▶ Mainly *epistemic significance*:
 - ▶ Not: What can an attacker do?
 - ▶ But: What do they reveal about the targetted system?

Adversarial attacks as a subject of study

- ▶ Not really significant security threat
- ▶ Mainly *epistemic significance*:
 - ▶ Not: What can an attacker do?
 - ▶ But: What do they reveal about the targetted system?

Before

- ▶ Accuracy as near-single metric
- ▶ Robustness assumed implicitly

Adversarial attacks as a subject of study

- ▶ Not really significant security threat
- ▶ Mainly *epistemic significance*:
 - ▶ Not: What can an attacker do?
 - ▶ But: What do they reveal about the targetted system?

Before

- ▶ Accuracy as near-single metric
- ▶ Robustness assumed implicitly

Afterwards

- ▶ High accuracy $\neq$ human-like understanding
- ▶ Robustness as property of interest

Key takeaways

- ▶ Imperceptible perturbation → wrong prediction
- ▶ Transferable across models, architectures, training data
- ▶ Categories: white/black-box · evasion/poisoning · targeted/untargeted
- ▶ Phenomenon still not fully explained
- ▶ Marginal security threat so far but epistemic relevance

Thank you for your attention!

Questions?



References I

- Goodfellow, Ian J, Jonathon Shlens, and Christian Szegedy (2014). “Explaining and harnessing adversarial examples”. In: *arXiv preprint arXiv:1412.6572*.
- Papernot, Nicolas, Patrick McDaniel, and Ian Goodfellow (2016). “Transferability in machine learning: from phenomena to black-box attacks using adversarial samples”. In: *arXiv preprint arXiv:1605.07277*.
- Szegedy, Christian et al. (2013). “Intriguing properties of neural networks”. In: *arXiv preprint arXiv:1312.6199*.
- Madry, Aleksander et al. (2017). “Towards deep learning models resistant to adversarial attacks”. In: *arXiv preprint arXiv:1706.06083*.
- Moosavi-Dezfooli, Seyed-Mohsen, Alhussein Fawzi, and Pascal Frossard (2016). “Deepfool: a simple and accurate method to fool deep neural networks”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2574–2582.
- Papernot, Nicolas, Patrick McDaniel, Ian Goodfellow, et al. (2017). “Practical black-box attacks against machine learning”. In: *Proceedings of the 2017 ACM on Asia conference on computer and communications security*, pp. 506–519.
- Ilyas, Andrew et al. (2019). “Adversarial examples are not bugs, they are features”. In: *Advances in Neural Information Processing Systems* 32.

References II

- Carlini, Nicholas and David Wagner (2017). “Towards evaluating the robustness of neural networks”. In: *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, pp. 39–57.
- Sharif, Mahmood et al. (2016). “Accessorize to a Crime: Real and Stealthy Attacks on State-of-the-Art Face Recognition”. In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 1528–1540.
- Xu, Kaidi et al. (2020). “Adversarial T-Shirt! Evading Person Detectors in a Physical World”. In: *Computer Vision – ECCV 2020*. Springer, pp. 665–681. DOI: [10.1007/978-3-030-58558-7_39](https://doi.org/10.1007/978-3-030-58558-7_39).
- Eykholt, Kevin et al. (2018). “Robust Physical-World Attacks on Deep Learning Visual Classification”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1625–1634.
- Athalye, Anish et al. (2018). “Synthesizing robust adversarial examples”. In: *International conference on machine learning*. PMLR, pp. 284–293.

White-box attacks to improve transferability

- ▶ Adversarial examples can differ in transferability!
- ▶ MI-FGSM: adds momentum to FGSM
- ▶ Ensemble attacks

ZOO: Proposed loss function

Black-box · Evasion · Targ./Untarg.

- ▶ Reminder: Carlini-Wagner loss: $\ell(x') = \max(\max_{i \neq t} z_i(x') - z_t(x'), 0)$
- ▶ Targeted ZOO: $l(x, t) = \max\{\max_{i \neq t} \{\log[f(x)]_i\} - \log[f(x)]_t, -\kappa\}$
- ▶ Untargeted ZOO: $l(x) = \max\{\log[f(x)]_y - \max_{i \neq y} \{\log[f(x)]_i\}, -\kappa\}$
- ▶ *log* to estimate logits

Backup Slides

Why that works

Ilyas et al. 2019

	Robust features point at	Non-robust features point at	Label	Image
Original clean training image (in $\mathcal{D}$)	dog	dog	dog	
Perturbed training image (in $\hat{\mathcal{D}}_{NR}$)	dog <i>(unaffected by perturbation)</i>	cat <i>(through adversarial perturbation)</i>	cat	
Original clean test image (in $\mathcal{D}$)	cat	cat	cat (predicted)	

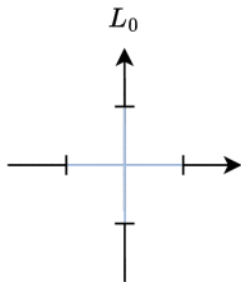
What is an Adversarial Example?

Definition (Adversarial Example)

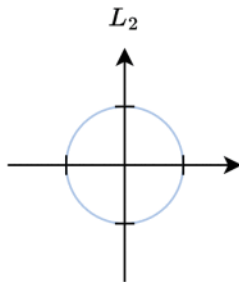
An *adversarial example* is an **input to a machine learning model** that has been intentionally perturbed, typically **in a subtle and often imperceptible manner**, such that the model **produces an incorrect prediction** while the semantic content of the input remains unchanged.

- ▶ **Machine learning models:** No quirk of deep networks, but a general property of machine learners
- ▶ **Subtle and imperceptible perturbations:** mathematically small under a chosen norm / visually imperceptible to humans
- ▶ **Produces an incorrect prediction:** The machine learner is fooled where the human would not have been

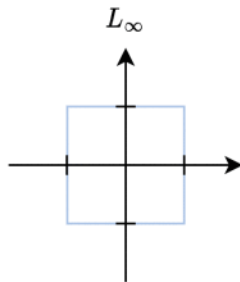
Measuring “small”: the ℓ_p norms



$$\|x\|_0 = (|x_1|^0 + \dots + |x_n|^0)$$



$$\|x\|_2 = (|x_1|^2 + \dots + |x_n|^2)^{\frac{1}{2}}$$



$$\|x\|_\infty = \max_i |x_i|$$

Norm defines the **perturbation budget** ϵ .

$$\underbrace{\min_{\delta} c \|\delta\|_2 + L_f(x + \delta, t)}_{\text{L-BFGS}} \longrightarrow \underbrace{\min_{\delta} \|\delta\|_p + c \cdot \ell(x + \delta)}_{\text{C\&W}}$$

- ▶ $\ell(x') = \max(\max_{i \neq t} z_i(x') - z_t(x'), 0)$
- ▶ Built so that $\ell(x') \leq 0 \iff$ input is classified as $t \rightarrow$ faithfully encodes the attack goal
- ▶ c decided through **binary search**

$$x' = \frac{1}{2}(\tanh(w) + 1), \quad w \in \mathbb{R}^m$$

- ▶ Mapping any $w \in \mathbb{R}^m$ into $(0, 1)^m \rightarrow$ every w decodes to a **valid image** $\rightarrow$ dissolving box-constraint
- ▶ Optimise w **unconstrained** with Adam optimiser

Significance

- ▶ Strongest attack of its time across $\ell_0/\ell_2/\ell_\infty$
- ▶ Broke *defensive distillation*, exposing it as *gradient masking*

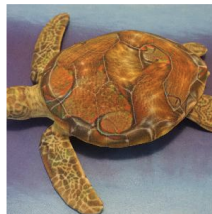
Physical adversarial attacks



Sharif et al. 2016 · Xu et al. 2020



Eykholt et al. 2018



Athalye et al. 2018

- ▶ Goal: get ML model to misclassify one specific sample v into target class t
- ▶ Assumption: model based on pretrained weights known to the attacker, access to data distribution
- ▶ Restriction: adversary cannot choose labels
- ▶ -! "Clean-label attack"

- ▶ Sample instance b of target class t
- ▶ Poison instance(s):

$$p = \arg \min_x \|g(x) - g(v)\|_2^2 + \lambda \|x - b\|_2^2$$

- ▶ g : feature representation (second-to-last layer output)
- ▶ Keep x close to v in feature space, but close to b in pixel space